CSE 327 P4

Team Members: Denny Li (del226@lehigh.edu)

Disclaimer: The results for `std` and `ming` may differ since during the run, the test and train query sets are not the exact same, so the results are not directly comparable for some of the results and the runs. However, we can still make some general observations about the relative performance of both methods based on the results obtained.

There are several notebooks, some located in its respective folder, where you can see its output. The rest are in the root directory, since they are shared across multiple runs.

4. Which performed better (std vs ming) (`countries` folder)?

The `std` method performed better than the `ming` method for the `countries`. While `ming` had one extra success vs `std`, overall, `std` on average generated less average nodes from the 100 queries (23,400 vs 101,400). Some of the hardest queries are those that require multiple conditions to be fulfilled and so the depth of the search tree is very large. For example: `sameRegion(City, C), sharesLanguage(C, france), threeBorderTrip(City, south_sudan), fourBorderTrip(C, south_sudan)`. While neither is able to solve it, `std` is able to "fail" faster. The total time for `std` is 49 seconds while `ming` is 1,540 seconds.

Overall, `std` is more efficient in terms of node generation and time taken, while `ming` may find a solution in some cases where `std` fails, but at the cost of generating many more nodes and taking more time.

6. For each random KB, compare the performance of `std` and `ming`. Which performed better (`size200`, `size300`, `size400`)?

- For the KB of size 200: `ming` performed better than `std`. `ming` had 39 standalone successes vs `std` single success. Compared to the `countries` KB, `ming` was able to solve it with less average nodes generated (40,500 vs 439,500).
- For the KB of size 300: `ming` performed better than `std`, only in terms of less average nodes generated (29 vs 82).
- For the KB of size 400: `std` performed better than `ming`. `std` had 3 standalone successes vs `ming`'s 0. Similar to the `countries` KB, `std` was able to solve it with less average nodes generated (40 vs 43,000).

Overall, it seems that as the size of the KB increases, `std` performs better than `ming`. This is likely because `std` is more efficient in terms of node generation and time taken, while `ming` may try to find a non-existant solution in some cases where `std` fails fast, but at the cost of generating many more nodes and taking more time.

7. (`size200`, `size300`, `size400`) Does increasing embedding size make it better or worse training loss? Does it improve or worsen test performance? Why?

> Note that the queries are not the same.

Increasing the embedding size generally leads to better training loss, as it drops lower faster, comparing the guided training loss at step 100.

- For `size200`, `ming` was able to solve one more query that it previeously failed vs the size 50 embedding.
- For `size300`, there was not much of a difference, but `ming` did generate slightly less average nodes.
- For `size400`, while `std` did take the lead, `ming` was able to generate much less average nodes than the size 50 embedding (from 43,000 to 27,700), and solved 2 more queries than the size 50 embedding.
- For `countries`, increasing the embedding size did allow it to surpass `std` in terms of both less average nodes generated (from 101,400 to 29,700 for `ming` vs 37,700 for `std` comparing the 64 size test queries), and solved one additional query.

Overall, increasing the embedding size seems to improve the performance of `ming`, as it allows `ming` to generate fewer nodes and solve more queries. This is likely because `ming` relies more on the quality of the embeddings to guide its search, and as embedding size increases, it can capture more complex relationships in the data, and unlike variables will be driven farther apart in the embedding space.

9. Evaluate the results of changing the architecture of the network.

The modified architecture is not consistently better than the default size-50 setup.

- On the `countries` KB, the modified architecture did not perform better than the `std` method, but relatively, it did generate less average nodes, from generating ~5x more nodes to 0.25x more with the new architecture.
- On the random KBs, the results are mixed:
  - `size200_mod`: the new architecture is worse than the original architecture.
  - `size300_mod`: the new architecture did not make as much of a difference.
  - `size400_mod`: the new architecture performed much better than the original architecture, being able to generate now less average nodes than `std`.

The modified architecture seems to produce mixed results, and it does seem to improve only when the size of the KB is large enough.

10. Create a table that summarizes the results of the experiments.

In [11]:
```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("reasoner_comparison_summary.csv")


def condition_label(folder):
    if folder.endswith("_mod"):
        return "modified"
    if folder.endswith("64"):
        return "embed64"
    return "default"


df["condition"] = df["folder"].map(condition_label)

# Relative improvement of learned reasoner over standard reasoner.
# Positive = learned reasoner is better (fewer nodes).
df["mean_node_reduction_pct"] = (
    (df["std_mean_nodes"] - df["ming_mean_nodes"]) / df["std_mean_nodes"]
) * 100

df["median_node_reduction_pct"] = (
    (df["std_median_nodes"] - df["ming_median_nodes"]) / df["std_median_nodes"]
) * 100

# Success rates for each reasoner, using the paired counts.
df["std_success_rate"] = (df["both_success"] + df["std_only_success"]) / df["queries"]
df["ming_success_rate"] = (df["both_success"] + df["ming_only_success"]) / df["queries"]

df["success_rate_diff_pct"] = (df["ming_success_rate"] - df["std_success_rate"]) * 100

# Optional: a simple win rate summary
df["learned_win_rate"] = df["ming_wins"] / df["queries"]
df["std_win_rate"] = df["std_wins"] / df["queries"]
```

```python
# Keep the most useful columns
df = df[
    [
        "folder",
        "condition",
        "std_mean_nodes",
        "ming_mean_nodes",
        "mean_node_reduction_pct",
        "std_median_nodes",
        "ming_median_nodes",
        "median_node_reduction_pct",
        "std_success_rate",
        "ming_success_rate",
        "success_rate_diff_pct",
        "std_win_rate",
        "learned_win_rate",
        "std_wins",
        "ming_wins",
        "ties",
        "both_success",
        "std_only_success",
        "ming_only_success",
        "both_fail",
    ]
]

df
```

Out[11]:

	folder	condition	std_mean_nodes	ming_mean_nodes	mean_node_reduction_pct	std_median_nodes	ming_median_nodes	median_node_reduction_pct	std_success_rate	ming_success_rate	success_rate_diff_pct	std_win_rate	learned_win_rate	std_wins	ming_wins	ties	both_success	std_only_success	ming_only_success	both_fail
0	countries	default	23357.36	101418.58	-334.203951	8.5	9.0	-5.882353	0.88	0.89	1.0	0.44	0.11	44	11	45	88	0	1	11
1	countries64	embed64	37721.32	29768.82	21.082242	4.5	4.5	0.000000	0.90	0.92	2.0	0.28	0.16	28	16	56	90	0	2	8
2	countriesmod	default	42575.78	58771.11	-38.038833	10.5	23.5	-123.809524	0.88	0.91	3.0	0.42	0.17	42	17	41	88	0	3	9
3	initial	default	874.92	3717.05	-324.844557	4.0	4.0	0.000000	0.99	0.99	0.0	0.10	0.22	10	22	68	99	0	0	1
4	size200	default	439454.22	40520.07	90.779456	18.0	3.0	83.333333	0.57	0.95	38.0	0.03	0.53	3	53	44	56	1	39	4
5	size200	default	439454.22	32457.35	92.614168	18.0	3.0	83.333333	0.57	0.96	39.0	0.04	0.55	4	55	41	56	1	40	3
6	size200_mod	modified	256241.14	117905.14	53.986647	8.0	5.0	37.500000	0.71	0.88	17.0	0.17	0.41	17	41	42	70	1	18	11
7	size300	default	82.73	28.61	65.417624	4.0	3.0	25.000000	1.00	1.00	0.0	0.21	0.35	21	35	44	100	0	0	0
8	size300	default	82.73	29.39	64.474798	4.0	3.0	25.000000	1.00	1.00	0.0	0.14	0.38	14	38	48	100	0	0	0
9	size300_mod	modified	214407.72	50251.03	76.562864	3.5	3.0	14.285714	0.84	0.95	11.0	0.03	0.33	3	33	64	83	1	12	4
10	size400	default	39.37	42997.57	-109114.046228	4.0	4.0	0.000000	1.00	0.97	-3.0	0.13	0.12	13	12	75	97	3	0	0
11	size400	default	39.37	27670.96	-70184.378969	4.0	5.0	-25.000000	1.00	0.99	-1.0	0.47	0.09	47	9	44	99	1	0	0
12	size400_mod	modified	107500.89	2676.65	97.510114	5.0	4.5	10.000000	0.90	1.00	10.0	0.08	0.19	8	19	73	90	0	10	0

11. Reflect on what you have learned through these experiments. Which versions of the learned model worked best? What sorts of differences do you see in performance across different KBs?

Using the relative improvement numbers, there is no single learned-model version that works everywhere. The best version depends on the KB.

The learned model was KB-dependent.

- For `countries`, the `embed64` version worked best: it was the only learned variant that clearly beat the standard reasoner on average node count. This is due to the fact that the `countries` KB is large and deeply interconnected.
- For `initial` (game of thrones), the learned model did not help much with the default setup, and the standard reasoner was better.
- For `size200`, both learned variants helped a lot, but `embed64` was slightly better.
- For `size300`, the `modified architecture` was best.
- For `size400`, the `modified architecture` was by far the best. The default learned model performed very poorly there, while the modified one was better.

Overall, changing the embedding size does help especially for larger KBs, especially for `countries`. Some architecture changes can help improve performance, but since I don't have the best background in creating optimal neural networks, I believed that the results are a bit of a coin toss. However, the default model is clearly not better in the majority of the cases because it is too simple and too small.